PHASE 12 OF 18

File I/O, NIO.2 & Serialization

Streams, the File class, byte vs character streams, buffering, charsets, PrintWriter, RandomAccessFile, java.nio.file (Path & Files), properties files, serialization, transient, serialVersionUID and safe alternatives

IN THIS PHASE

12.1 Java I/O Overview
12.2 The File Class (java.io)
12.3 Byte Streams
12.4 Character Streams and Buffering
12.5 Choosing the Right Class
12.6 Other Useful java.io Classes
12.7 NIO.2: Path and Files (Java 7+)
12.8 Properties Files: External Configuration
12.9 Serialization
12.10 Best Practices
12.11 Predict the Output
12.12 Interview Questions
12.13 Quick Revision

12.1 Java I/O Overview

I/O (Input/Output) means reading data from a **source** (keyboard, file, network socket, memory) and writing it to a **destination**. Java models this as **streams**: sequences of data flowing in one direction.

```
Source --> InputStream/Reader --> Java program --> OutputStream/Writer --> Destination
(file, keyboard, socket, byte[])                    (file, console, socket, byte[])
```

Package	Contents
`java.io` (Java 1.0)	Classic stream-based I/O: `File`, `InputStream`/`OutputStream`, `Reader`/`Writer`, serialization
`java.nio` (Java 1.4)	Buffers, channels, selectors: non-blocking, high-performance I/O
`java.nio.file` (NIO.2, Java 7)	Modern file API: `Path`, `Files`, `FileSystems`, file walking and watching. **Preferred for files today.**

Byte streams vs character streams

	Byte streams	Character streams
Unit	8-bit bytes	16-bit chars (text), decoded using a **charset**
Abstract base classes	`InputStream`, `OutputStream`	`Reader`, `Writer`
Use for	**Binary data**: images, PDFs, ZIPs, audio, serialized objects	**Text**: .txt, .csv, .json, .xml, logs
File classes	`FileInputStream`, `FileOutputStream`	`FileReader`, `FileWriter`
Buffered versions	`BufferedInputStream`, `BufferedOutputStream`	`BufferedReader`, `BufferedWriter`
Bridge between them	n/a	`InputStreamReader` (bytes → chars), `OutputStreamWriter` (chars → bytes)

```
InputStream (abstract)                      Reader (abstract)
 +- FileInputStream                          +- InputStreamReader
 +- ByteArrayInputStream                     |   +- FileReader
 +- FilterInputStream                        +- BufferedReader
 |   +- BufferedInputStream                  +- StringReader, CharArrayReader
 |   +- DataInputStream
 +- ObjectInputStream                       Writer (abstract)
                                             +- OutputStreamWriter
OutputStream (abstract)                      |   +- FileWriter
 +- FileOutputStream                         +- BufferedWriter
 +- ByteArrayOutputStream                    +- PrintWriter
 +- FilterOutputStream                       +- StringWriter, CharArrayWriter
 |   +- BufferedOutputStream
 |   +- DataOutputStream
 |   +- PrintStream   (System.out, System.err)
 +- ObjectOutputStream
```

> **NOTE**
> Most of these classes follow the **Decorator pattern**: you wrap one stream in another to add features. `new BufferedReader(new InputStreamReader(new FileInputStream(f), UTF_8))` adds decoding and then buffering on top of raw bytes. Closing the **outermost** wrapper closes the whole chain.

12.2 The File Class (java.io)

A `File` object represents a **path** to a file or directory. It **does not read or write content**; it handles names, existence checks, metadata, and creating or deleting entries.

```java
File f = new File("abc.txt");                  // logical object; nothing is created yet
System.out.println(f.exists());                // false
System.out.println(f.createNewFile());         // true  -> empty file created
System.out.println(f.createNewFile());         // false -> it already exists

File dir = new File("data");
dir.mkdir();                                   // one level; false if exists/no parent
new File("data/2026/reports").mkdirs();        // creates all missing parent directories
File inDir = new File(dir, "test.txt");        // parent + child
inDir.createNewFile();

for (File item : dir.listFiles()) {            // list(): names; listFiles(): File objects
    System.out.println((item.isFile() ? "File : " : "Directory : ") + item.getName());
}
```

Method	Returns	Purpose
`exists()`, `isFile()`, `isDirectory()`	`boolean`	Existence and type checks
`createNewFile()`	`boolean`	Create an empty file; `false` if it exists; throws `IOException`
`mkdir()` / `mkdirs()`	`boolean`	Create a directory / with parents
`delete()`	`boolean`	Delete a file or an **empty** directory (`false` otherwise)
`list()` / `listFiles()`	`String[]` / `File[]`	Directory contents (`null` if not a directory!)
`length()`	`long`	Size in bytes
`getName()`, `getPath()`, `getAbsolutePath()`, `getParent()`	`String`	Path information
`renameTo(File)`	`boolean`	Rename or move (platform-dependent)
`canRead()`, `canWrite()`, `lastModified()`	various	Permissions and metadata

TIP

`File` methods return `boolean` instead of throwing a meaningful exception, so a failed `delete()` doesn't tell you *why*. NIO.2 `Files` methods throw descriptive exceptions (`NoSuchFileException`, `DirectoryNotEmptyException`), which is one reason to prefer them.

12.3 Byte Streams

```
// Copy any file (binary-safe) using a buffer
try (InputStream in = new FileInputStream("photo.jpg");
     OutputStream out = new FileOutputStream("photo-copy.jpg")) {
    byte[] buffer = new byte[8192];
    int bytesRead;
    while ((bytesRead = in.read(buffer)) != -1) {     // read() returns -1 at end of stream
        out.write(buffer, 0, bytesRead);              // write only the bytes actually read
    }
}
// Java 9+: in.transferTo(out) does the same loop for you

try (FileOutputStream log = new FileOutputStream("audit.log", true)) {   // true = APPEND
    log.write("login ok\n".getBytes(StandardCharsets.UTF_8));
}
```

PITFALL

`read()` (no arguments) returns **one byte as an** `int` (0–255), or **-1** at end of stream. Reading and writing one byte at a time without buffering is extremely slow, because each call can hit the OS. Use a `byte[]` buffer or `Buffered*` streams.

12.4 Character Streams and Buffering

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("customers.txt"))) {
    bw.write("101,Rohit,Pune");
    bw.newLine();                                  // platform line separator
    bw.write("102,Amit,Delhi");
}                                                  // close() flushes the buffer

try (BufferedReader br = new BufferedReader(new FileReader("customers.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {       // readLine() returns null at end of file
        String[] parts = line.split(",");
        System.out.println(parts[1] + " lives in " + parts[2]);
    }
}
```

Character encodings (charsets)

Text is stored as bytes, and a **charset** maps between characters and bytes. Reading with the wrong charset produces garbled text: ₹ or é shows up as â‚¹ or Ã©.

```
// Explicit charset (Java 11+ constructors): always predictable
var cs = StandardCharsets.UTF_8;
try (BufferedReader br = new BufferedReader(new FileReader("in.txt", cs));
     BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt", cs))) {
    bw.write(br.readLine());
}

// Bridge classes: bytes <-> chars with a chosen charset (works on any InputStream)
Reader r = new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8);
Writer w = new OutputStreamWriter(new FileOutputStream("out.txt"), StandardCharsets.UTF_8);
```

> **NOTE**
> Before **Java 18**, `FileReader`/`FileWriter` used the **platform default** charset (often windows-1252 on Windows), a classic "works on my machine" bug. Since Java 18 (JEP 400), the default is **UTF-8** everywhere. Specifying `StandardCharsets.UTF_8` explicitly is still best practice.

PrintWriter and PrintStream

```
try (PrintWriter pw = new PrintWriter(new FileWriter("statement.txt"))) {
    pw.println("ACCOUNT STATEMENT");
    pw.printf("%-12s %10s%n", "Date", "Amount");
    pw.printf("%-12s %,10.2f%n", "2026-08-01", 25000.0);
}
System.out.println("...");      // System.out is a PrintStream
System.err.println("error!");   // so is System.err
```

> **PITFALL**
> `PrintWriter` and `PrintStream` **never throw** `IOException`; they set an internal error flag instead. Call `checkError()` if you need to know whether writes failed. Forgetting to flush or close is the usual cause of an "empty file".

12.5 Choosing the Right Class

Comparison	Key difference
`Reader` vs `InputStream`	Characters (decoded text) vs raw bytes
`Writer` vs `OutputStream`	Characters vs raw bytes
`BufferedReader` vs `FileReader`	`BufferedReader` buffers (fast) and has `readLine()`. `FileReader` reads unbuffered, one character or array at a time. Wrap one in the other.
`BufferedWriter` vs `FileWriter`	Buffered (fewer system calls) and has `newLine()`; must be flushed or closed
`BufferedReader` vs `Scanner`	`BufferedReader`: fast, 8 KB buffer, lines only, throws `IOException`. `Scanner`: parses tokens and primitives (`nextInt`), slower, 1 KB buffer, swallows `IOException`.
`PrintWriter` vs `PrintStream`	Character stream (Writer family) vs byte stream (`System.out`). Both have `printf`/`println` and neither throws `IOException`.
`java.io.File` vs `java.nio.file.Files`	Old path object with boolean results vs modern utilities with clear exceptions, charsets, streams and walking

12.6 Other Useful java.io Classes

Data streams: primitives in binary

```
try (DataOutputStream out = new DataOutputStream(
        new BufferedOutputStream(new FileOutputStream("rates.bin")))) {
    out.writeInt(3); out.writeDouble(7.25); out.writeUTF("FD");
}
try (DataInputStream in = new DataInputStream(
        new BufferedInputStream(new FileInputStream("rates.bin")))) {
    int n = in.readInt(); double rate = in.readDouble(); String type = in.readUTF();
    // must read in EXACTLY the same order and types as written
}
```

RandomAccessFile: jump to any position

```
try (RandomAccessFile raf = new RandomAccessFile("records.dat", "rw")) {  // r, rw, rws, rwd
    raf.writeInt(100); raf.writeInt(200); raf.writeInt(300);   // 4 bytes each
    raf.seek(4);                              // move the file pointer to the 2nd int
    System.out.println(raf.readInt());        // 200
    raf.seek(4); raf.writeInt(250);           // update in place
    System.out.println(raf.length() + " bytes, pointer at " + raf.getFilePointer());
}
```

Memory streams and Console

```
ByteArrayOutputStream bos = new ByteArrayOutputStream();   // write into memory
bos.write("hello".getBytes());
byte[] bytes = bos.toByteArray();
InputStream in = new ByteArrayInputStream(bytes);         // great for unit tests

Console console = System.console();                       // null inside most IDEs!
if (console != null) {
    String user = console.readLine("User: ");
    char[] pwd = console.readPassword("Password: ");      // input not echoed
    java.util.Arrays.fill(pwd, ' ');                      // wipe the secret after use
}
```

12.7 NIO.2: Path and Files (Java 7+)

`java.nio.file` is the modern, recommended API for working with files: concise, charset-aware, with clear exceptions and stream support.

```
import java.nio.file.*;
import static java.nio.charset.StandardCharsets.UTF_8;

Path dir  = Path.of("data", "reports");              // Java 11 (Paths.get(...) before that)
Path file = dir.resolve("summary.txt");              // data/reports/summary.txt

Files.createDirectories(dir);                        // like mkdirs; no error if it exists
Files.writeString(file, "Total: 1,25,000\n", UTF_8);                    // Java 11
Files.writeString(file, "Count: 42\n", UTF_8, StandardOpenOption.APPEND);
String content = Files.readString(file, UTF_8);                         // whole file
List<String> lines = Files.readAllLines(file, UTF_8);                   // all lines

try (Stream<String> stream = Files.lines(file, UTF_8)) {   // lazy: for HUGE files
    long count = stream.filter(l -> l.startsWith("Total")).count();
}                                                          // Files.lines must be closed!

Files.copy(file, dir.resolve("backup.txt"), StandardCopyOption.REPLACE_EXISTING);
Files.move(dir.resolve("backup.txt"), dir.resolve("old.txt"), StandardCopyOption.ATOMIC_MOVE);
Files.deleteIfExists(dir.resolve("old.txt"));
System.out.println(Files.exists(file) + " " + Files.size(file) + " bytes");
```

`Files` method	Purpose
`readString` / `writeString` (11)	Whole file as one String
`readAllLines` / `write(path, lines)`	All lines as a `List<String>`
`readAllBytes` / `write(path, bytes)`	Binary content
`lines(path)`	Lazy `Stream<String>` (close it: try-with-resources)
`newBufferedReader` / `newBufferedWriter`	Buffered character streams with a charset
`newInputStream` / `newOutputStream`	Byte streams
`exists`, `notExists`, `isDirectory`, `isRegularFile`, `isReadable`, `size`	Checks and metadata

`Files` method	Purpose
`createFile`, `createDirectory`, `createDirectories`, `createTempFile`	Creation
`copy`, `move`, `delete`, `deleteIfExists`	Manipulation (`StandardCopyOption.REPLACE_EXISTING`, `ATOMIC_MOVE`)
`list(dir)`, `walk(dir)`, `find(dir, depth, matcher)`	Directory listing and recursive traversal (Streams: close them)
`getLastModifiedTime`, `getAttribute`, `probeContentType`	Attributes

`StandardOpenOption`	Meaning
`CREATE` / `CREATE_NEW`	Create if missing / fail if it already exists
`APPEND`	Write at the end
`TRUNCATE_EXISTING`	Clear the existing content (the default for write methods along with `CREATE`, `WRITE`)
`READ` / `WRITE`	Access mode

Path operations

```
Path p = Path.of("/home/rohit/projects/../docs/./notes.txt");
p.getFileName();                 // notes.txt
p.getParent();                   // /home/rohit/projects/../docs/.
p.normalize();                   // /home/rohit/docs/notes.txt
p.isAbsolute();                  // true
Path base = Path.of("/home/rohit");
base.resolve("docs/a.txt");      // /home/rohit/docs/a.txt
base.relativize(Path.of("/home/rohit/docs/a.txt"));   // docs/a.txt
Path.of("report.pdf").toAbsolutePath();               // resolved against the working directory
p.toFile();  new File("x").toPath();                  // convert between old and new APIs
```

Practical example: summarise a transactions CSV

```
// transactions.csv:  txnId,account,type,amount
//                    T1,SB-1001,CREDIT,5000.00
//                    T2,SB-1001,DEBIT,1200.50
Path csv = Path.of("transactions.csv");
BigDecimal balance = BigDecimal.ZERO;
try (BufferedReader br = Files.newBufferedReader(csv, UTF_8)) {
    br.readLine();                                                  // skip the header
    String line;
    while ((line = br.readLine()) != null) {
        String[] f = line.split(",", -1);
        BigDecimal amt = new BigDecimal(f[3].trim());
        balance = f[2].equals("CREDIT") ? balance.add(amt) : balance.subtract(amt);
    }
}
System.out.println("Net: " + balance);                           // Net: 3799.50

try (Stream<Path> files = Files.walk(Path.of("logs"))) {         // recursive search
    files.filter(f -> f.toString().endsWith(".log"))
         .filter(f -> f.toFile().length() > 1_000_000)
         .forEach(System.out::println);                         // large log files
}
```

12.8 Properties Files: External Configuration

`.properties` files hold `key=value` configuration. Test frameworks (Appium/Selenium settings), Spring Boot's `application.properties` and JDBC setups all use them.

```
# config.properties
app.env=QA
device.name=Pixel_7
platform.version=14
api.base.url=https://qa.bank.example.com
```

```
Properties props = new Properties();
try (Reader r = Files.newBufferedReader(Path.of("config.properties"), UTF_8)) {
    props.load(r);
}
String env = props.getProperty("app.env");                    // QA
String timeout = props.getProperty("timeout.seconds", "30");   // default if missing

// Load from the classpath (src/main/resources or src/test/resources)
try (InputStream in = Config.class.getClassLoader().getResourceAsStream("config.properties")) {
    props.load(in);
}
```

> **TIP**
> Load config from the **classpath** (`getResourceAsStream`) rather than a hard-coded path like `C:\Users\...`. It then works the same in your IDE, in Maven/Gradle and in CI. Keep secrets out of committed properties files; use environment variables or a vault.

12.9 Serialization

Serialization converts an object's state into a **byte stream** (to save to a file, send over a network, or cache). **Deserialization** rebuilds the object from those bytes.

```
 Object (heap) --ObjectOutputStream.writeObject()--> bytes --> file / network / cache
 Object (heap) <--ObjectInputStream.readObject()---- bytes <--
```

```
import java.io.*;

public class Customer implements Serializable {        // marker interface: no methods
    private static final long serialVersionUID = 1L;   // version id: declare it explicitly!
    private String id;
    private String name;
    private transient String password;                 // transient: NOT serialized
    private static String bank = "ABC";                // static: class-level, NOT serialized

    public Customer(String id, String name, String password) {
        this.id = id; this.name = name; this.password = password;
    }
    @Override public String toString() { return id + " " + name + " " + password; }
}

Customer c = new Customer("C1", "Rohit", "secret");
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("customer.ser"))) {
    oos.writeObject(c);                                // serialize
}
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("customer.ser"))) {
    Customer copy = (Customer) ois.readObject();       // deserialize (ClassNotFoundException)
    System.out.println(copy);                          // C1 Rohit null  <- transient lost
}
```

Rules

Rule	Detail
Must implement `Serializable`	Otherwise `NotSerializableException`
Whole object graph	Every non-transient field's object must also be serializable (a `Customer` with an `Address` field needs `Address implements Serializable`)
`transient` fields	Skipped. They get default values (null, 0, false) after deserialization. Use for passwords, caches, derived data and non-serializable helpers.
`static` fields	Not serialized: they belong to the class, not the object
Constructor	**Not called** for the serializable class when deserializing. Fields come from the stream.
Non-serializable parent	Its **no-arg constructor runs** during deserialization, and its fields are re-initialised rather than restored. The parent **must** have an accessible no-arg constructor.
Arrays and collections	Serializable if their elements are (`ArrayList`, `HashMap` and `String` are)

serialVersionUID

Each serializable class has a version number. During deserialization the JVM compares the stream's UID with the current class's UID; if they differ, it throws `InvalidClassException`. If you don't declare it, the compiler **computes** one from the class structure, so even adding a method can break compatibility with previously saved data. **Always declare** `private static final long serialVersionUID = 1L;` and change it deliberately.

Customising serialization

```
public class Account implements Serializable {
    private static final long serialVersionUID = 1L;
    private String accountNo;
    private transient String pin;                    // never write the raw PIN

    @Serial
    private void writeObject(ObjectOutputStream out) throws IOException {
        out.defaultWriteObject();                    // normal fields
        out.writeObject(mask(pin));                  // custom extra data
    }
    @Serial
    private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
        in.defaultReadObject();
        String masked = (String) in.readObject();
        // validate invariants here; re-create transient helpers
    }
    private static String mask(String p) { return p == null ? null : "****"; }
}
```

`Externalizable` (with `writeExternal`/`readExternal` and a **mandatory public no-arg constructor**) gives you full manual control. A singleton class should implement `readResolve()` to return its instance, otherwise deserialization creates a second object (enum singletons handle this automatically).

> **PITFALL**
> **Never deserialize untrusted data** with `ObjectInputStream`. Crafted byte streams can trigger remote code execution through "gadget" classes on the classpath. This has caused major real-world vulnerabilities (Apache Commons Collections, CVE-2015-7501). If you must, use an `ObjectInputFilter` (Java 9+) allow-list. For data exchange, prefer **JSON** (Jackson, Gson), XML, or Protocol Buffers: they are language-neutral, human-readable or compact, and safer.

```
// Typical modern approach (Jackson library, used by Spring Boot)
ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(customer);          // {"id":"C1","name":"Rohit"}
Customer back = mapper.readValue(json, Customer.class);
```

12.10 Best Practices

1. Always use **try-with-resources** for streams, readers, writers and `Files.lines`/`walk`/`list` streams.
2. Prefer `java.nio.file.Files` for everyday file work; fall back to `java.io` streams for streaming or legacy APIs.
3. **Buffer** I/O (`Buffered*`, `Files.newBufferedReader`), never byte-by-byte.
4. **Specify the charset** (`UTF_8`) whenever converting between bytes and text.
5. Stream large files line by line (`readLine`, `Files.lines`) instead of loading them whole.
6. Build paths with `Path.of(a, b)` / `resolve` instead of concatenating `"\\"` or `"/"`, and load config from the classpath.
7. Handle `NoSuchFileException`, `AccessDeniedException` and `FileAlreadyExistsException` explicitly where it matters.
8. In tests, use **temporary files and folders** (`Files.createTempFile`, JUnit 5 `@TempDir`) and clean up.
9. Declare `serialVersionUID`, mark secrets `transient`, and **avoid Java serialization for untrusted input**; use JSON.

12.11 Predict the Output

```
File f = new File("demo-dir");
System.out.println(f.mkdir() + " " + f.mkdir());      // true false
System.out.println(new File("missing-dir").list());   // null (not a directory)

Path p = Files.writeString(Path.of("t.txt"), "a\nb\nc");
System.out.println(Files.readAllLines(p).size());     // 3
System.out.println(Files.size(p));                    // 5 (bytes: a \n b \n c)

try (BufferedReader br = Files.newBufferedReader(p)) {
    br.readLine(); br.readLine(); br.readLine();
    System.out.println(br.readLine());                // null (end of file)
}

ByteArrayInputStream in = new ByteArrayInputStream(new byte[]{65, 66});
System.out.println(in.read() + " " + in.read() + " " + in.read());   // 65 66 -1

Path base = Path.of("/a/b");
System.out.println(base.resolve("c/../d").normalize());   // /a/b/d
```

12.12 Interview Questions

Q. What is the difference between byte streams and character streams?

Byte streams (`InputStream`/`OutputStream`) handle raw 8-bit data and suit binary files. Character streams (`Reader`/`Writer`) handle text, decoding bytes into characters with a charset.

Q. Why use BufferedReader and BufferedWriter?

They reduce expensive system calls by reading and writing large chunks into an in-memory buffer. They also add `readLine()` and `newLine()`.

Q. What is the difference between FileReader and InputStreamReader?

`InputStreamReader` is a bridge that converts **any** byte stream to characters with a chosen charset. `FileReader` is a convenience subclass for files; before Java 11 it couldn't take a charset and used the platform default.

Q. What is the difference between BufferedReader and Scanner?
`BufferedReader` is faster with a bigger buffer and reads lines only. `Scanner` parses tokens and primitives with regex, is slower, and hides `IOException`. Use `BufferedReader` for large input and `Scanner` for convenient parsing of small input.

Q. What is the difference between `java.io.File` and `java.nio.file.Files`/`Path`?
`File` is the legacy path abstraction with boolean-returning methods and limited features. NIO.2 offers `Path` plus `Files` utilities with clear exceptions, charsets, symbolic links, attributes, stream-based reading and directory walking.

Q. What is serialization? How do you make a class serializable?
Converting an object to a byte stream for storage or transfer. Implement the marker interface `Serializable` (all non-transient fields must be serializable too) and use `ObjectOutputStream.writeObject()`.

Q. What is `transient`? Are static fields serialized?
`transient` fields are excluded from serialization and get default values when restored. Static fields belong to the class, so they are never serialized.

Q. What is serialVersionUID, and why should you declare it?
A version identifier checked during deserialization; a mismatch throws `InvalidClassException`. If you don't declare it, it is computed from the class structure, so harmless changes can break compatibility.

Q. Is a constructor called during deserialization?
Not for the serializable class itself. The no-arg constructor of the first **non-serializable** superclass is called, so that superclass's fields are re-initialised rather than restored.

Q. What are the risks of Java serialization?
Deserializing untrusted data can execute malicious gadget chains (RCE). Serialization also creates fragile versioning, bypasses constructors and invariants, and is Java-only. Prefer JSON or Protobuf, or use `ObjectInputFilter`.

Q. How do you read a very large file efficiently?
Stream it line by line with `BufferedReader.readLine()` or `Files.lines()` inside try-with-resources, rather than `readAllLines`/`readString`, which load everything into memory.

12.13 Quick Revision

- Streams flow one way. Bytes: `InputStream`/`OutputStream`; text: `Reader`/`Writer`; bridges: `InputStreamReader`/`OutputStreamWriter`.
- Decorator pattern: wrap streams to add buffering or encoding; closing the outer one closes the chain.
- `File` = a path plus metadata (no content). `createNewFile`/`mkdir(s)`/`delete`/`listFiles` return booleans or `null`.
- `read()` returns -1 at EOF; `readLine()` returns `null` at EOF. Always buffer.
- Specify **UTF-8**. Java 18+ defaults to UTF-8, but older platforms didn't.
- `PrintWriter`/`PrintStream` never throw `IOException` (use `checkError`); flush or close!
- NIO.2: `Path.of`, `resolve`, `normalize`; `Files.readString`/`writeString`/`readAllLines`/`lines`/`copy`/`move`/`walk`. Close `lines`, `walk` and `list` streams.
- Properties: `props.load(reader)`, `getProperty(key, default)`; load from the classpath.
- Serialization: `Serializable` (marker), `ObjectOutputStream.writeObject`, `ObjectInputStream.readObject`.
- `transient` and `static` are not serialized. Declare `serialVersionUID`. Mismatch → `InvalidClassException`.

- No constructor runs for the serializable class on read; a non-serializable parent needs a no-arg constructor.
- Never deserialize untrusted bytes; prefer JSON (Jackson).